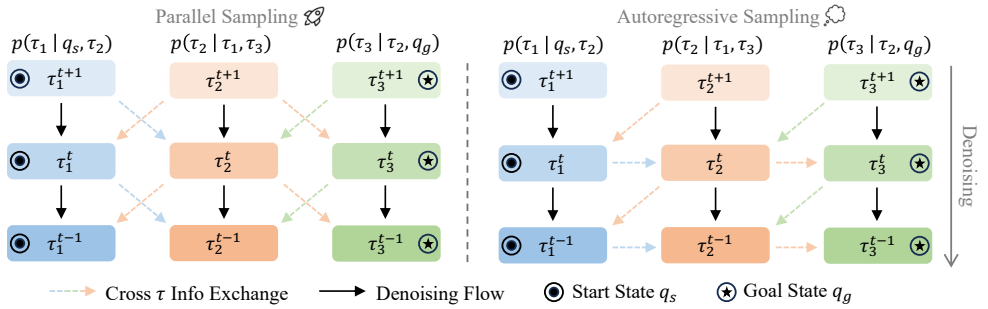

Figure 3: **Compositional Trajectory Planning: Parallel Sampling and Autoregressive Sampling**. We present an illustrative example of sampling three trajectories $\tau_{1:3}$ with the proposed compositional sampling methods. Dashed lines represent cross trajectory information exchange between adjacent trajectories and black lines represent the denoising flow of each trajectory. In parallel sampling, $\tau_{1:3}$ can be denoised concurrently; while in autoregressive sampling, denoising τ_k depends on the previous trajectory τ_{k-1}, e.g., the denoising of τ_2 depends on τ_1 (as shown in the blue horizontal dashed arrows). Additionally, start state q_s and goal state q_g conditioning are applied to the trajectories in the two ends, τ_1 and τ_3, which enables goal-conditioned planning. Trajectories $\tau_{1:3}$ will be merged to form a longer plan τ_{comp} after the full diffusion denoising process.

values of neighboring chunks. As a result, to sample from the composed distribution, one would need a blocked Gibbs sampling procedure, where the value of each individual trajectory chunk is iteratively sampled given decoded values of neighboring trajectory chunks. This sampling procedure is slow, and passing information across chunks to form a consistent plan is challenging.

Information Propagation Through Noisy-Sample Conditioning. We propose a more efficient approach that addresses these challenges by leveraging the progressive denoising process of diffusion models. The key challenge in composing trajectories is ensuring feasible transitions between each pair of neighboring chunks, i.e., maintaining physical constraints and dynamic consistency at connection points where segments overlap. Our key insight is that we can achieve this by having trajectory segments guide each other's generation during the diffusion process: as one segment takes shape through denoising, it helps shape its neighbors into compatible configurations.

We implement this insight using a diffusion model that generates trajectory chunks conditioning on their neighbors' *noisy samples*. Given a dataset D of trajectories τ, we train a denoising network ϵ_θ to learn the trajectory distribution $p_\theta(\tau_k|\tau_{k-1}, \tau_{k+1})$ with the training objective

$$\mathcal{L}_{\text{nbr}} = \mathbb{E}_{\tau\in D,t,k}\left[\|\epsilon - \epsilon_\theta(\tau_k^t, t \mid \tau_{k-1}^t, \tau_{k+1}^t)\|^2\right], \tag{3}$$

where k identifies a trajectory segment, t is the noise level, and τ_k^t represents segment k corrupted with noise level t. Crucially, when denoising each segment, the network conditions on noisy versions of neighboring segments $\tau_{k-1}^t, \tau_{k+1}^t$ at the same noise level. This allows each segment to influence its neighbors' denoising process, ensuring their final configurations are dynamically compatible. In addition, further training the network to condition on τ_{k-1}^{t-1} can enable autoregressive compositional sampling, which we will discuss in Section 3.3. In practice, we only need to condition on the small overlapping regions between consecutive trajectories, making the generation process efficient while maintaining consistency across connection points.

Representing Initial States and Goals. In addition, we further train the same denoising network to represent the distributions $p_\theta(\tau_1|q_s, \tau_2)$ and $p_\theta(\tau_K|\tau_{K-1}, q_g)$. This corresponds to the training objective

$$\mathcal{L}_{\text{start}} = \mathbb{E}_{\tau\in D,t,k}\left[\|\epsilon - \epsilon_\theta(\tau_1^t, t \mid q_s, \tau_2^t)\|^2\right], \tag{4}$$

with an analogous objective for the conditioned goal state q_g. We train the same denoising network ϵ_θ with both conditioning. Please see Appendix E for implementation details. We provide an overview of our proposed training strategy in Algorithm 1.

3.3 Compositional Trajectory Planning

Our compositional framework enables flexible sampling strategies for generating long-horizon plans with Equation 2. The basic sampling process starts by initializing each trajectory chunk τ_k with Gaussian noise. Then, through iterative denoising, each chunk is denoised while being conditioned